Annexe A17 — L'adaptation des fichiers JSON

LibraryV3 · Phase G, étape 3.1 à 3.3d et 3.5b · la règle d'écriture, JsonReader, les décisions D1 à D9, les scènes régénérées

Résultat en une ligne. Un fichier de scène est désormais un **document complet** : toute clé que le moteur lit y est écrite, soit avec sa valeur, soit avec `null` pour déléguer au moteur. Une absence n'est plus jamais muette. Les 10 scènes v1 et les fichiers de configuration se chargent à **zéro souci**, et le système solaire n'a plus qu'**une seule source** : `gen_solar_system.py`.

1. La règle (Onky, 7 octobre 2026)

La première proposition distinguait clés « requises » (avertissement si absentes) et clés « optionnelles » (silence). Onky l'a refusée : **si une clé est absente, c'est soit un oubli, soit un choix, et dans les deux cas il faut signaler la valeur prise par le moteur.** Un `ReadOr` silencieux aurait rendu « l'auteur a oublié » et « l'auteur a délégué » impossibles à distinguer.

Ce que l'auteur écrit	Sens	Ce que le moteur affiche	Niveau
une valeur exacte	valeur de l'auteur	rien	—
`null`	délégation **écrite**	« `= null` … — valeur retenue : X »	**info** (blanc, non compté)
clé absente	souci (oubli ou non)	« absente … — défaut pris : X »	**warn** (jaune, compté)
type invalide	erreur de l'auteur	« de type invalide … — défaut pris : X »	**warn**

Deux choix de niveau, et leur raison :

- **`null` en info, pas en warn.** C'est une annonce, pas un souci. Le compteur `Logger::warnCount()` ne doit compter que les soucis : c'est lui qui prouve qu'un chargement est « à zéro souci ».
- **Clé absente en warn, pas en error.** Le rouge reste réservé à « chargement impossible » (un `id` absent, un parent introuvable). Une clé absente reçoit un défaut connu et affiché, et le chargement continue.

> **À retenir.** `null` a remplacé la sentinelle `-1`. Avec une sentinelle, un `-1` voulu et un `-1` écrit par erreur sont indiscernables. Avec `null`, l'intention est **présente** dans le fichier. C'est la même règle que `hasFarPlane + farPlane` : un booléen (ou une absence explicite) et une valeur, jamais une valeur magique.

2. JsonReader : cinq verbes, une seule règle

Verbe	Usage	absente	`null`	mauvais type
`Read(key, def)`	valeur simple	souci + `def`	info + `def`	souci + `def`
`TryRead(key, valeurMoteur)`	**surcharge** d'une valeur moteur (caméra → `EngineConfig`)	souci, `nullopt`	info, `nullopt`	souci, `nullopt`
`ReadVector(key, def)`	tableau de **exactement** 3 nombres	souci	info	souci (« mal formée »)
`Delegate(key, sens)`	la clé est **consommée** ici, son contenu sera lu par un autre parseur (`parent`, `components`, `nodes`)	souci	info	—

Verbe	Usage	absente	`null`	mauvais type
`Child(key)`	sous-bloc	souci, lecteur vide	info, lecteur **délégué**	souci, lecteur vide

Le lecteur délégué (`Child` sur un bloc `null`) annonce chaque clé qu'il ne trouve pas en info, avec la mention « (bloc null) », au lieu d'en faire un souci : l'auteur a écrit « tout le bloc par défaut », il serait faux de lui reprocher chaque clé.

Deux règles complètent les verbes :

- **R28** : `find()`, jamais `operator[]`. Sur un json non const, `operator[]` **insère** un `null` ; sur un json const, une clé absente est un comportement indéfini.
- **`WarnUnread()`** à la fin de chaque lecteur signale toute clé écrite mais jamais lue. Le préfixe `_` marque une clé de **documentation**, assumée et non lue (`_type`, `_note`, `_range`) : elle n'est jamais signalée.

```
// Read : la forme de reference (JsonReader.h)
template<typename T> [[nodiscard]] T Read(const char* key, T def)
{
    m_seen.insert(key);                                  // pour WarnUnread
    const auto it = m_j.find(key);                       // R28
    if (it == m_j.end()) { Absent(key, ToLog(def));          return def; }
    if (it->is_null())   { AnnounceDefault(key, ToLog(def)); return def; }
    try { return it->get<T>(); }
    catch (const nlohmann::json::type_error&) { WarnDefault(key, "de type invalide",
ToLog(def)); return def; }
}
```

> × Lire un même objet avec **deux** lecteurs (par exemple une passe 1 pour les composants, une passe 2 pour `parent`) et appeler `WarnUnread` dans chacun : chaque passe signalerait comme « ignorées » les clés lues par l'autre. **Un objet est lu par un seul lecteur, d'un bout à l'autre.** C'est pourquoi `parent` est désormais *lu* en passe 1 et *résolu* en passe 2 (`ResolveParents`), sans seconde lecture du JSON.

2.1 Les surcharges : valider à l'entrée

`depthDisplayRange` et `lodTolerancePx` sont des **surcharges** de valeurs d'`engine.json`, par caméra. `ReadPositiveOverride` lit avec `TryRead`, puis applique le domaine (`> 0` et fini) **au parsing**. Le composant porte `bool m_hasX` + `float m_X`, et `BuildViewData` choisit sur le booléen puis **asserte** l'invariant.

Pourquoi tester « fini » alors que JSON n'a pas de `NaN` ? Parce que `1e39` est un nombre JSON valide : nlohmann le stocke en `double`, puis `get<float>()` le convertit hors domaine, ce qui donne `+inf` sur MSVC. La garde s'écrit sous forme positive niée, `!(v > 0)`, qui rejette aussi un éventuel NaN.

2.2 À quoi sert `depthDisplayRange`

C'est un **réglage d'affichage de debug** (règle R20), utilisé seulement en mode `Depth` : la distance, en unités monde, qui correspond au noir. Avec 80 (la valeur d'`engine.json`), tout ce qui est au-delà de l'orbite de Mars est noir ; `Top_View`, à 950 unités, aurait un écran entièrement noir. D'où une valeur par caméra : `150` dans les scènes du système solaire.

3. Les décisions de format (D1 à D9)

#	Décision	Pourquoi
D1	les racines écrivent `"parent": null`	une racine est une intention ; `parent` absent est un souci

#	Décision	Pourquoi
D2	`"type"` (679 nœuds) devient `_type`	une étiquette pour l'auteur, que le moteur ne lit pas ; s'il la lit un jour, on retire le `_` et le diff le montre
D3	`texture` est **supprimée** des scènes	une texture est un paramètre de **matériau**, pas d'instance ; elle passe par le MTL (A18) ; la surcharge de matériau par instance viendra en L08
D4	`"POINT_LIGHT"` devient `"Point"` ; `rangeUnits` devient `_range`	le vocabulaire du parseur ; la portée ne sera lue qu'en L10, et aucune autre distance ne porte de suffixe d'unité
D5	triggers explicites : `"role": "zone"`, évènements `""`	`""` est une **valeur exacte** (« aucun évènement »), pas une délégation
D6	`m_currentHealth` devient `currentHealth`, défaut = `maxHealth`	un format de scène ne porte jamais le nom d'un membre C++
D7	9 scènes v1 régénérées ; `solar_system_v2.json` exclu	le v2 est le schéma cible d'un futur Serializer et ne se charge pas aujourd'hui
D8	bloc absent : un souci pour le bloc, puis un par clé	on obtient la liste complète des valeurs prises
D9	un MTL par **asset**, partagé par toute sa chaîne de LOD	A18 § 2

3.1 La portée d'une lumière (D4), en avance sur L10

Une source ponctuelle s'atténue en 1/d² et ne s'annule jamais. La **portée** est une coupure d'ingénierie (Unity `Light.range`, Unreal `AttenuationRadius`) : elle permet d'écarter une lumière d'un objet dont la boîte ne touche pas sa sphère, et elle donne l'échelle de la loi d'atténuation. La valeur 600 couvre Eris (494 unités).

> × Couper brutalement : `f(d) = 1/d²` si `d < range`, sinon 0. Juste avant `range`, `f` vaut 1/range², pas zéro : on obtient un cercle visible sur les objets qui traversent la frontière. La forme standard multiplie par une fenêtre qui tend doucement vers zéro : `f(d) = (1/d²) · saturate(1 − (d/range)⁴)²`. Elle sera étudiée en L10.

4. Le niveau fichier et nœud (pas 3.3a)

Toute la lecture du fichier et des nœuds se faisait **en nlohmann brut**, hors de la règle. Quatre défauts graves y vivaient : `sceneName` absent terminait le programme, `id` absent était un comportement indéfini, une faute `"parnet"` transformait un nœud en racine sans un mot, et le tableau `nodes` était lu deux fois. Depuis :

```
JsonReader rs(sceneData, "Scene", jsonSceneFile);
const std::string sceneName = rs.Read("sceneName", std::string("<sans nom>"));
const nlo_json* pNodes = rs.Delegate("nodes", "aucun noeud");
rs.WarnUnread();
if (!pNodes || !pNodes->is_array()) { /* scene refusee */ return false; }
// PASSE 1 : chaque noeud lu UNE fois, par UN lecteur (id, parent delegue, components delegue)
// PASSE 2 : ResolveParents, plus aucune lecture JSON
```

Un **changement de comportement voulu** : une scène sans `nodes` était chargée vide avec un avertissement ; elle est maintenant **refusée**. Une scène vide reste possible, mais elle s'écrit : `"nodes": []`. Un graphe faux ne se charge pas « presque bien ».

5. Les scènes régénérées (pas 3.3d)

Chaque clé lue par le Serializer est **écrite**. Une clé qui manquait reçoit **la valeur que le moteur prenait déjà**, pour que le comportement ne change pas. La vérification a porté sur l'état du moteur, pas sur le texte : chaque scène a été chargée deux fois (originale puis normalisée), et l'état de **chaque entité** a été relevé (parent, Transform, Mesh, caméra, contrôleurs, Trigger, Health, Light). Une seule différence sur les 9 scènes, attendue : la lumière du Soleil passe de `Ambient` à `Point` (D4).

Les tableaux de nombres sont écrits sur une ligne ; les scènes générées passent de 430 Ko à 315 Ko.

6. Une seule source pour le système solaire

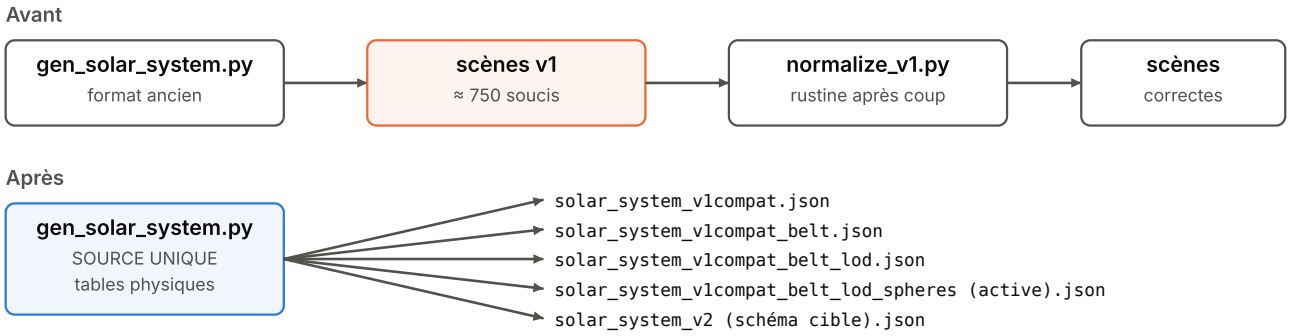


Figure 1 — Une scène n'est jamais retouchée à la main : on corrige le générateur, puis on régénère. La migration (normalize) a servi une fois et n'a plus de rôle pour le système solaire.

`gen_solar_system.py` part des **tables physiques** (rayons, demi-grands axes, périodes, obliquités) et produit les cinq scènes. Il avait pris du retard : ancien format, pas de variantes LOD (dérivées à la main pendant le chantier LOD). Il a été corrigé à la source plutôt que réparé après coup :

Correction	Détail
ancre de l'**étoile**	rattachée à `Solar-System` sans compensation d'échelle ; sinon toutes les orbites seraient divisées par le rayon du Soleil
Mesh	plus de `texture` ; `orbitalSpeed` et `rotationSpeed` toujours écrits (0.0 = immobile)
évènements de trigger	traduits dans le vocabulaire du moteur pour le v1 (`v1_events`) ; le v2 garde les noms descriptifs
caméras	un seul jeu (`_camera()`), identique dans les 5 scènes ; `far` écrit seulement si `infiniteFar` vaut `false` ; caméra orthographique : `infiniteFar` impossible, donc `far: 2000`
`Player_Ship`	`"role": "probe"` : avec `"zone"`, il ne déclenchait aucune zone (paires zone-zone ignorées)
sorties	5 fichiers, dont les deux variantes LOD par `with_lod()` ; format d'écriture commun

Critère de réussite : les scènes produites ont été comparées **sémantiquement** à celles du dépôt. Les seules différences sont celles listées ci-dessus, et le chargement donne 0 souci.

> **À retenir.** Un générateur et un outil de migration n'ont pas le même rôle. Le générateur est le **plan de l'architecte** : on le relance à chaque changement de contenu. La migration est l'**ouvrier qui remet aux normes** les maisons déjà construites : elle sert une fois. Le nom d'un outil est sa promesse : `gen_solar_system_v2.py` aurait annoncé « génère », « système solaire » et « v2 », trois promesses fausses pour un outil de migration.

7. Les fichiers de configuration

Fichier	Changement
`engine.json`	`depthDisplayRange` et `tolerancePx` validés ; blocs `debug`, `lod`, `simulationclock`, `viewport` lus par `Child` (plus de `if Has` silencieux) ; nouvelle clé `lod.pointRadiusPx` (étape 4, A20)
`config.json`	la lecture du bloc `configuration` (`REP_OBJ_DEFAULT`, `REP_GFX_DEFAULT`), inutilisé par tout le code, est **supprimée**

8. Les scènes de test (pas 3.5b)

Les quatre scènes écrites à la main étaient déjà au bon format ; leurs défauts étaient autour : meshes absents du dépôt, MTL issus de 3ds Max (`Tf` non lu, texture en chemin absolu `C:\…\fishy.pcx`), et un fichier mélangé qui avait **dérivé** de sa référence (9 valeurs). `solar_system.json` est devenu `scene_test.json` ; `solar_system_shuffled.json` a été reconstruit à partir de lui, et le chargement des deux donne une hiérarchie **identique** sur les 17 entités (R26).

9. Aide-mémoire

Question	Réponse
Je veux la valeur du moteur	écrire la clé à `null`
Une clé ne doit pas être lue	la préfixer de `_`
Une racine	`"parent": null`
Un trigger sans évènement	`""` explicite
Une caméra à plan lointain infini	ne pas écrire `far`
Pas de gizmo	`"gizmo": null` ou `"length": 0`
Modifier le système solaire	modifier `gen_solar_system.py`, régénérer, jamais à la main
Vérifier une scène	chargement : `warnCount` doit rester à 0 ; les lignes blanches listent les délégations